

QUIZZY

Software Requirements Specification

AI-Powered Real-Time Multiplayer Quiz Platform

Document Information	Details
Document Type	Software Requirements Specification (SRS)
System	Quizzzy
Version	1.0
Status	Deployed / Live
Prepared For	Academic / Project Documentation
Date	October 2026

Prepared by: ___Engr. Tayyaba Sadaqat___

Table of Contents

1. Introduction
2. Overall Description
3. System Architecture
4. Functional Requirements
5. External Interface Requirements
6. Non-Functional Requirements
7. Data Requirements
8. Use Cases
9. Business Rules and Game Logic
10. Error Handling
11. Security and Privacy Requirements
12. Deployment Requirements
13. Acceptance Criteria
14. Future Enhancements
15. Conclusion

1. Introduction

1.1 Purpose

This Software Requirements Specification defines the functional and non-functional requirements of Quizzzy, an AI-powered real-time multiplayer quiz platform. The document describes the behavior expected from the host and player interfaces, quiz creation workflows, AI-assisted quiz generation, multiplayer game flow, scoring, elimination, leaderboard and podium behavior, and deployment architecture.

1.2 Scope

Quizzzy enables a host to create or generate a multiple-choice quiz, configure gameplay settings, create a game room, and share a numeric game PIN. Players join the room using the PIN and a display name. During a live session, questions and answers are synchronized between host and players through WebSocket communication. Quizzzy supports question and answer shuffling, focus-mode behavior, elimination rules, score/streak tracking, live leaderboards, and a final podium. An AI Quiz Maker uses the Groq API to generate structured quizzes from a topic, difficulty, question count, and optional instructions.

1.3 Intended Audience

- Students and instructors using Quizzzy
- Project supervisors/evaluators
- Developers maintaining the system
- Testers validating functional and deployment requirements

1.4 Definitions and Abbreviations

Term	Meaning
SRS	Software Requirements Specification
Host	User who creates/configures and controls a live quiz session
Player	Participant who joins a live game using a game PIN
Game PIN	Numeric room identifier used to join a hosted game
AI Quiz Maker	Feature that generates quiz content through the Groq API
WebSocket	Persistent bidirectional connection used for live multiplayer events
API	Application Programming Interface
CORS	Cross-Origin Resource Sharing
Focus Mode	Gameplay setting intended to detect/handle loss of focus or tab switching

2. Overall Description

2.1 Product Perspective

Quizzzy is a web-based client/server application. The user-facing application is implemented with Next.js, while a FastAPI backend manages game rooms, real-time communication and AI quiz generation. The production frontend is hosted on Vercel and the backend is hosted on PythonAnywhere. Groq provides the external AI model API.

2.2 User Classes

User	Primary Responsibilities
Host	Create or generate quizzes, configure settings, create a room, monitor players, start/advance/reveal/finish the game, and view results.
Player	Join with a valid PIN, enter a name, answer questions, receive score/status updates, and view final results.

2.3 Operating Environment

- Modern desktop and mobile web browsers with JavaScript enabled.
- Internet connectivity is required for deployed multiplayer gameplay and AI generation.
- Frontend: Next.js 16.x application.
- Backend: Python/FastAPI served by Uvicorn/ASGI.
- Real-time channel: secure WebSockets (WSS) in production.
- AI service: Groq chat-completions API.

3. System Architecture

The system follows a separated frontend/backend architecture. The Next.js client sends HTTP requests to FastAPI for room creation and AI generation. Live host/player events are carried over WebSocket connections. The backend maintains active game state and invokes Groq only from the server side so the API key is not exposed to browsers.

Layer	Technology / Responsibility
Presentation	Next.js, React, Framer Motion; host and player interfaces
HTTP API	FastAPI endpoints for game creation and AI quiz generation
Real-Time	FastAPI WebSocket endpoints for host/player synchronization
Game Engine	Server-side game manager for players, questions, scoring, elimination and game state
AI	Groq API; production model selected from available compatible text models
Frontend Hosting	Vercel
Backend Hosting	PythonAnywhere ASGI/Uvicorn

4. Functional Requirements

ID	Requirement	Specification
FR-01	Manual Quiz Creation	The system shall allow a host to create a quiz with a title and multiple-choice questions.
FR-02	Question Definition	Each quiz question shall contain question text, four answer options, one correct answer index and a time limit.
FR-03	AI Quiz Generation	The system shall allow a host to generate a quiz by providing a topic, difficulty, number of questions and optional instructions.
FR-04	AI Output Validation	The backend shall validate AI-generated quiz JSON before returning it to the frontend, including question count, four non-empty options, valid correct-answer index and allowed time limit.
FR-05	Game Creation	The host shall be able to create a live game room from the current quiz.

FR-06	Game PIN	The backend shall generate a game PIN for a newly created room and make it available to the host.
FR-07	Player Join	A player shall be able to join an existing game using a valid game PIN and a display name.
FR-08	Lobby	The host lobby shall display the current list and count of connected players.
FR-09	Start Game	The host shall be able to start the game when at least one player is present and the host connection is active.
FR-10	Question Delivery	The server shall deliver the active question state to connected players during the live game.
FR-11	Answer Submission	A player shall be able to submit one answer for the current question.
FR-12	Scoring	The backend shall calculate and maintain player scores and streak-related state based on submitted answers.
FR-13	Question Reveal	The host shall be able to reveal the result/state of the current question.
FR-14	Next Question	The host shall be able to advance the live game to the next available question without leaving host and player states out of sync.
FR-15	Question Shuffle	When enabled, the system shall randomize the question order for the game session.
FR-16	Answer Shuffle	When enabled, the system shall randomize answer-option order while preserving the mapping to the correct answer.
FR-17	Focus Mode	When enabled, the player interface shall monitor focus/visibility behavior according to the configured game rules.
FR-18	Elimination	When a player is eliminated, the server shall mark/remove that player from active competition and broadcast the elimination event.
FR-19	Eliminated Player Visibility	Eliminated players shall no longer appear in the active host player list, leaderboard or final podium.
FR-20	Elimination Notification	The host/game interface shall be able to display that a named player has been eliminated.
FR-21	Leaderboard	The system shall provide a leaderboard containing eligible active players ordered by game performance.
FR-22	Finish Game	The host shall be able to finish the game after gameplay is complete.
FR-23	Podium	The system shall present final ranked results/podium for eligible players at the end of the game.
FR-24	Disconnection Handling	The backend shall handle host/player WebSocket disconnections without corrupting the room state.
FR-25	Error Messaging	The system shall return/display meaningful error information when game creation, AI generation, joining or live-game actions fail.

5. External Interface Requirements

5.1 User Interface

- Host interfaces shall provide quiz creation, AI generation, lobby, live-game controls and podium/results views.
- Player interfaces shall provide PIN/name entry, waiting state, question/answer interaction and finished-game state.
- Interactive controls shall visibly communicate disabled states, connection/waiting states and gameplay transitions.
- The interface shall be responsive enough for common desktop and mobile browser use.

5.2 HTTP Interfaces

Endpoint	Method	Purpose
/api/games	POST	Create a live game room from quiz/settings data.
/api/games/{pin}	GET	Retrieve game information required by the client.
/api/ai/generate-quiz	POST	Generate and validate an AI-created quiz.

5.3 WebSocket Interfaces

Interface	Purpose
/ws/host/{pin}	Maintains the host's real-time connection and receives host control events.
/ws/player/{pin}	Maintains a player's real-time connection and receives player events/answers.

5.4 Third-Party Interface

Groq is accessed only by the FastAPI backend. The GROQ_API_KEY shall remain a server-side secret. The deployed PythonAnywhere environment routes outbound Groq HTTP traffic through its required proxy configuration.

6. Non-Functional Requirements

ID	Category	Requirement
NFR-01	Usability	Core host and player flows should be understandable without technical knowledge.
NFR-02	Responsiveness	The web interface should adapt to common desktop and mobile viewport sizes.
NFR-03	Real-Time Behavior	Gameplay state changes should be propagated through WebSockets with minimal practical delay.
NFR-04	Reliability	A valid host action shall not intentionally advance only one side of the game; server state is authoritative for synchronization.
NFR-05	Security	Secrets such as GROQ_API_KEY shall never be embedded in NEXT_PUBLIC variables or client-side source.
NFR-06	Transport Security	Production HTTP and WebSocket traffic shall use HTTPS/WSS endpoints.
NFR-07	Maintainability	Frontend, API and game-management responsibilities should remain separated to simplify maintenance.
NFR-08	Compatibility	The application should work on current Chromium-based browsers and other modern standards-compliant browsers.
NFR-09	Validation	AI-generated and user-provided game data shall be validated before being used in live gameplay.
NFR-10	Availability	The application depends on the availability and limitations of Vercel, PythonAnywhere and Groq services.

7. Data Requirements

7.1 Quiz Data

A quiz contains a title and a collection of questions. Each question contains a unique identifier where applicable, text, four options, a zero-based correct-answer index and an allowed time limit.

7.2 Game Session Data

- Game PIN and quiz configuration
- Connected/active player records
- Current question index and game phase
- Player answers, scores and streak state
- Shuffle/focus/elimination settings
- Leaderboard and final ranking state

7.3 AI Generation Input

Field	Description
topic	Subject of the requested quiz; required.
difficulty	Requested difficulty such as Easy, Medium or another supported value.
count	Requested question count; backend constrains the value to a supported range.
instructions	Optional additional generation guidance.

8. Use Cases

Use Case	Actor(s)	Main Flow	Postcondition
UC-01: Create Quiz Manually	Host	Host opens quiz creation, enters title/questions/options/correct answers/time limits, and prepares the quiz for hosting.	A valid quiz is available for game creation.
UC-02: Generate Quiz with AI	Host	Host enters topic, difficulty, count and optional instructions; frontend sends request; backend calls Groq; backend validates structured output; generated quiz is returned.	A validated quiz is available for review/hosting.
UC-03: Create and Join Room	Host, Player	Host creates a room and receives PIN. Player enters PIN and name. Backend validates room and adds player. Lobby updates in real time.	Player appears in the host lobby.
UC-04: Play Live Quiz	Host, Player	Host starts game. Server broadcasts question state. Player answers. Backend evaluates answer and updates score. Host reveals/advances until complete.	Game progresses consistently for connected participants.
UC-05: Eliminate Player	System	When elimination criteria are met, the server updates player eligibility, broadcasts the event and excludes the eliminated player from active rankings.	Eliminated player is removed from active lists/leaderboard/podium.
UC-06: Finish and Show Podium	Host, System	Host finishes the game. Backend finalizes standings and clients transition to result views.	Eligible top players are shown in final ranking/podium.

9. Business Rules and Game Logic

- A game cannot be started from the lobby when no players are present.
- A player answer is associated with the currently active question and player session.
- The correct answer must remain logically correct even when answer options are shuffled.
- Question shuffling changes game order only when the corresponding setting is enabled.
- Eliminated players are not eligible for subsequent active-player lists, leaderboard positions or podium positions.
- The host controls major game-state transitions such as start, reveal, next question and finish.
- The backend is the authoritative source of live multiplayer state.

10. Error Handling

Condition	Expected Behavior
Missing/invalid game PIN	Player is prevented from joining and receives a meaningful error.
No players in lobby	Start Game remains unavailable.
WebSocket connection problem	Client communicates connection state/failure and avoids sending actions on a closed socket.
Groq key unavailable	AI endpoint returns a clear configuration error without exposing secrets.
Groq/API request rejected	Backend returns a sanitized error message to the frontend.
Malformed AI output	Backend rejects invalid quiz structure rather than starting a game with corrupt data.
Invalid host transition	Backend returns/broadcasts an error instead of silently corrupting game state.

11. Security and Privacy Requirements

- The Groq API key shall remain in the backend environment and shall not be exposed through client-side environment variables.
- Only public backend base URLs may be stored in NEXT_PUBLIC_API_URL and NEXT_PUBLIC_WS_URL.
- Production communication shall use HTTPS and WSS.
- CORS shall allow the approved production frontend origin and required local-development origins.
- Application error responses shall not intentionally reveal API keys or other secret configuration values.
- Display names and gameplay state should be limited to data necessary for the live quiz experience.

12. Deployment Requirements

Component	Production Requirement
Frontend	Next.js production build deployed on Vercel.
Backend	FastAPI application deployed as an ASGI/Uvicorn website on PythonAnywhere.
Frontend API URL	NEXT_PUBLIC_API_URL points to the public HTTPS backend origin.
Frontend WebSocket URL	NEXT_PUBLIC_WS_URL points to the public WSS backend origin.
AI Secret	GROQ_API_KEY exists only in the backend environment/.env and is never committed/exposed intentionally.

Groq Networking	PythonAnywhere deployment uses its outbound proxy for the Groq HTTP client where required.
-----------------	--

13. Acceptance Criteria

- Host can create a valid game and receives a PIN.
- A second browser/device can join the live room using the PIN.
- Host sees the joined player in the lobby.
- Starting the game moves connected participants into the live quiz flow.
- Players can submit answers and the server updates scoring state.
- Next-question transitions keep host and player views synchronized.
- Question shuffling works when enabled.
- Answer shuffling preserves the correct-answer mapping.
- Focus mode behavior activates when enabled according to browser capabilities.
- An eliminated player is removed from active host lists, leaderboard and podium, and an elimination message is shown.
- Game can finish without the 'Game could not be finished' failure.
- Final leaderboard/podium contains only eligible players.
- AI Quiz Maker generates a valid quiz from the deployed application.
- Production frontend communicates successfully with the deployed backend over HTTPS/WSS.

14. Future Enhancements

Possible future work may include persistent accounts and quiz libraries, analytics/history, richer moderation, additional question types, classroom/team modes, improved accessibility controls, stronger reconnect/session recovery, and optional database-backed persistence. These items are enhancements and are not requirements of the current documented release.

15. Conclusion

Quizzy provides an end-to-end browser-based multiplayer quiz experience combining manual quiz authoring, AI-assisted content generation and real-time competitive gameplay. This SRS establishes the expected behavior of the current system and provides a baseline for testing, evaluation, maintenance and future development.